

COMPLETE STANDALONE TRANSCRIPTION

AWS Domain 3 - Technology

25 source pages • typed from the uploaded handwritten source • no external additions

Source-faithful edition. This document rewrites the uploaded handwritten notes into clean typed text. It preserves the concepts, examples, figures, service names, comparisons, and practical notes **as written in the source**. No external material is added, and the notes are not externally verified or updated.

Source Page 1 - EC2, Compute, Pricing Options, Access Methods

Compute - Amazon EC2

- **EC2 = Elastic Compute Cloud.**
- Used to **rent and manage virtual servers.**
- EC2 instances are not considered serverless.
- EC2 instance uses a **preconfigured template: Amazon Machine Image (AMI).**
- Source note: approximately **750 compute hours free** in the free tier.

EC2 use cases written in the notes

1. Deploy a database and have **full control.**
2. Deploy a **web application.**

Access methods to EC2

1. **AWS Management Console.**
2. **Secure Shell (SSH)** - source note highlights access with key pair.
3. **EC2 Instance Connect (EIC).**
4. **AWS Systems Manager** - via browser-based shell or CLI.

Pricing options - based on use case and budget

1. On-Demand

- Pricing **down to the second** is emphasized.
- No upfront payment / no long commitment.
- Use cases written:
 - Application under development.
 - Unpredictable workload that cannot be interrupted.
 - Workload that will not run more than a year.

2. Spot

- Uses **unused EC2 capacity**.
- Application can run at a **very low price**.
- Suitable when work can **start/stop** and **can be interrupted**.
- Source note: can provide a very large discount compared with On-Demand (the handwritten note references around **90%**).

3. Reserved Instances

- Commit to a specific instance type for **1 or 3 years**.
 - Use cases written:
 - Application needs **capacity reservation**.
 - Upfront discount / reduced cost.
 - Steady-state usage.
-

Source Page 2 - Reserved Instances Continued, Load Balancing, Auto Scaling, Dedicated Hosts, Savings Plans

Reserved Instances - continuation

- The source notes mention a discount around **72%** with commitment / compute usage.
- **Capacity Reservation** is highlighted for situations where capacity must be guaranteed.
- Another source note mentions a discount around **75%**.
- Can reserve capacity in a specific **Availability Zone (AZ)**.
- Payment options written:
 - **All Upfront.**
 - **Partial Upfront.**
 - **No Upfront.**
- **Convertible Reserved Instance** type is noted; the source mentions a discount around **54%**.

1. Elastic Load Balancing (ELB)

- Distributes traffic between **multiple instances**.

Types drawn / listed in the notes:

Application Load Balancer

- HTTP/HTTPS traffic.
- **Layer 7.**
- Notes mention sticky/session-related behavior, DNS, and routing.

Classic Load Balancer

- Layers **4 & 7** are written.

Network Load Balancer

- High performance for **TCP/UDP**.
- **Layer 4.**
- Static IP is noted.

Gateway Load Balancer

- **Layer 3.**
- Routing and firewall use is noted.

2. EC2 Auto Scaling

- Add or replace instances **automatically based on demand.**
- Notes mention operation across Availability Zones.
- **Horizontal scaling (scale out):** increase the number of instances.
- **Vertical scaling (scale up):** increase capacity / size of the server.

4. Dedicated Host

- A **physical server.**
- Use cases written:
 - Compliance / multi-tenancy requirements.
 - Server-bound licenses from vendors such as Microsoft.
- Licensing examples written:
 - Per socket.
 - Per core.
 - Per VM.

Dedicated Instances

- The notes distinguish dedicated instances from dedicated hosts and mention VMs running on dedicated infrastructure.
- A handwritten percentage around **70%** appears next to the dedicated-instance note; retained here as written, without external validation.

5. Savings Plans

- Commit to a certain level of **compute usage per hour** for **1 or 3 years.**
 - Use cases / benefits written:
 - Flexibility to change **instance type** and **OS.**
 - Can apply across **multiple compute services.**
-

Source Page 3 - Lambda, Fargate, Lightsail

2. AWS Lambda

- **Serverless compute service.**
- Write/run code **without managing servers.**
- Lambda **scales automatically.**
- Application code is organized into **functions**, described as building blocks for serverless applications.

Lambda use cases

1. Real-time file processing:

- Upload files to S3.
- Trigger a Lambda function.
- Function performs an action.

2. Sending email notifications.

3. Backend business logic (example note mentions Alexa).

Lambda features / notes

- Supported languages written:
 - Java.
 - Go.
 - PowerShell.
 - Node.js.
 - C#.
 - Python.
 - Ruby.
- You can write code **locally and upload it**, or write it in the **Lambda console**.
- Lambda executes code **in response to an event**.
- Source note: a Lambda function has a **15-minute timeout**.

3. AWS Fargate

- **Serverless compute engine for containers.**
- Manages containers.
- Source explanation: put the application into a container and deploy/transport it as a unit; functionality remains isolated from the underlying server.
- Like Lambda, Fargate is described as **serverless** and **scales automatically**.

4. Amazon Lightsail

- Quickly launch resources for **small projects**.
 - Deploy preconfigured applications such as **WordPress**.
 - Simple screen/interface for people with little or no cloud experience.
 - Provides a low, predictable monthly fee; the source gives an example around **\$3.50**.
 - Bundled items written alongside Lightsail:
 - VMs.
 - SSD storage.
 - DNS management.
 - Static IP.
 - Data transfer.
 - Source note: described as similar in spirit to **AWS Quick Starts**.
-

Source Page 4 - S3, Outposts, Batch

Storage - 1. Amazon S3 (Simple Storage Service)

- A place to store data accumulated over years for analysis.
- **Object storage** service.
- Described as **highly available**.
- Objects/files are stored in **buckets** (directories in the handwritten explanation).
- S3 is described as **unlimited storage** that can hold millions of objects.
- Objects can be **private or public**.
- Upload objects through:
 - Console.
 - CLI.
 - Programmatically using SDKs.

S3 security controls written

Security can be set at the bucket level or individual-object level using:

- **ACLs.**
- **Bucket Policies.**
- **Access Point Policies.**

S3 Versioning

- Enable versioning to create multiple versions of an object.
- Benefits written:
 - Protect against accidental deletion.
 - Use/recover a previous version.

5. AWS Outposts

- Allows you to run **cloud services in your own data center / on-premises**.
- Used when data must remain on-premises due to:
 - Latency requirements.
 - Data-sovereignty / data-residency needs.
- Delivers / installs AWS servers in the data center so cloud services can be used in a **hybrid experience**.
- APIs can be used to develop on-premises applications using AWS-style services.

6. AWS Batch

- Used with large workloads that need processing.
 - Break the workload into **small chunks or batches** and process them in parallel.
 - Run hundreds or thousands of small batch-processing jobs.
 - Dynamically provisions instances based on volume.
-

Source Page 5 - S3 Properties, Storage Classes, Intelligent-Tiering

S3 cross-region replication

- You can set up **cross-region replication**.
- Source notes mention choosing the desired region rather than relying only on a default location.

S3 Storage Classes - choose based on budget + use case

1. S3 Standard

- General-purpose storage recommended for **frequently accessed data**.
- Data stored across **multiple AZs**.
- Offers **low latency** and **high throughput**.
- The source contrasts availability and durability values and emphasizes their difference.

2. S3 Intelligent-Tiering

- Uses automation / intelligence behind the scenes.
- Automatically moves data to the **most cost-effective storage class**.
- Described as an **automatic cost-saving** option.
- Recommended for data with **unknown or changing access patterns**.
- Notes mention:
 - No retrieval fees (as written in the source note).
 - No need for the user to manually manage retrieval / tier movement.
 - Data stored across multiple AZs.

S3 access logs

- Can use **access logs** to track access to buckets and objects.

Regional service / globally unique names

- S3 is a **regional service**.
- Bucket names are **globally unique**.

Durability

- Described as the ability to ensure objects are not lost or compromised.
- Source states S3 is designed for **11 9s of durability (99.999999999%)**.

Availability

- Described as the ability to access data when you need it.
 - Source states S3 is designed for **99.99% availability**.
 - Data inside a bucket is replicated / accessible through multiple servers within the same region, as described in the notes.
-

Source Page 6 - S3 Infrequent Access and Glacier Retrieval

3. S3 Standard-Infrequent Access (Standard-IA)

- For **less frequently accessed data** that still requires **rapid access**.
- Millisecond access when needed.
- Cheaper than Standard.
- Used for long-lived data.
- Data stored across **multiple AZs**.
- Source notes:
 - **11 9s durability**.
 - **99.9% availability**.

4. S3 One Zone-Infrequent Access (One Zone-IA)

- For less frequently accessed data that still requires rapid / millisecond access.
- Source note: approximately **20% cheaper than Standard-IA**.
- Stored in **one AZ**.
- Lower availability is noted compared with multi-AZ options.
- Appropriate when data is **re-creatable** and maximum availability/durability across multiple AZs is not essential.
- A highlighted source note states a **minimum storage duration**; the source comparison references the same minimum-duration concept used with IA classes.

5. Glacier Instant Retrieval

- For long-term data storage / archival at lower cost, such as long-term backups.
- Data retrieval in **milliseconds** when needed.
- Data stored across multiple AZs.
- Source emphasizes high durability.
- **Minimum storage duration: 90 days**.

6. Glacier Flexible Retrieval

- For long-term archived backups.
 - Three retrieval options written:
 1. **1-5 minutes - Expedited.**
 2. **3-5 hours - Standard.**
 3. **5-12 hours - Bulk (free).**
 - Data stored across multiple AZs with durability/availability noted.
 - **Minimum storage duration: 90 days.**
-

Source Page 7 - Glacier Deep Archive, S3 Outposts, S3 Use Cases

7. S3 Glacier Deep Archive

- Long-term archival data.
- Data may be accessed **once or twice a year**.
- Suitable for retaining data for **compliance requirements**.
- Described as the **cheapest option of all S3 options** in the source.
- Data retrieval options:
 1. **12 hours**.
 2. **48 hours**.
- Data stored across multiple AZs with the same durability / availability concept noted.
- **Minimum storage duration: 180 days**.

8. S3 on Outposts

- Provides **object storage on premises** for data that needs to remain local.
- Source explanation: if you want on-premises capabilities while retaining S3-style storage, use S3 on Outposts / Standard Storage Class as written.

S3 use cases

1. **Static websites**; use **CloudFront** for global distribution.
2. **Data archive using Glacier**.
3. Store data for analytics with **Athena & Redshift**.
4. Mobile applications uploading files; source notes using **S3 Transfer Acceleration** to speed the process.

Source reminder

- You can transition between S3 classes using **S3 Lifecycle**.
-

Source Page 8 - EC2 Storage, EBS, Instance Store

EC2 Storage

- All EC2 instances must have a **root drive**.
- Source identifies two storage choices around an EC2 instance:
 - **EBS**.
 - **Instance Store**.
- If storage needs to be shared and associated with many instances, the source points to **EFS**.

2. EBS - Elastic Block Store

- A device called a **volume** that can attach to an instance.
- Can attach to **one instance within the same AZ** (as written in the notes).
- Data **persists** even if the instance is stopped.
- **Tied to one AZ**.
- If data must exist in another AZ, the source notes that you must **replicate / recreate the block storage**.

3. Instance Store

- A **local store** physically attached to the host computer / physical server.
- Described as the **fastest option for data access** with higher I/O speeds.
- **Temporary / ephemeral**.
- If the instance is stopped/terminated, data can be lost.

Instance Store use cases

1. Temporary storage needs.
 2. Data replicated across multiple instances.
- Source note: because data can be replicated, this can be cheaper for some workloads.

EBS use cases

1. Store long-term production data that needs quick access; described as the most common storage service for EC2.
 2. Run a database on an instance using an EBS volume.
-

Source Page 9 - EFS, Storage Gateway, AWS Backup

4. Amazon EFS - Elastic File System

- A **serverless network file system** for sharing files.
- Source comparison: similar conceptually to a shared drive / Google Drive.
- Supports **Linux file system** workloads (as written).
- Accessible from **different AZs in the same region**.
- Source comparison:
 - EBS is tied to 1 AZ.
 - EFS is multi-AZ / shared and is **more expensive than EBS**.

EFS use cases

1. Main / shared directories for **business-critical applications**.
2. **Lift and shift** existing enterprise applications from on-premises to the cloud with minimal infrastructure changes.

5. AWS Storage Gateway

- A **hybrid storage service**.
- Used when you want some data in the cloud and some data on-premises with low latency.
- Connect on-premises data with cloud data using:
 - Site-to-site VPN.
 - Direct Connect.

Storage Gateway use cases

1. Moving backups to the cloud.
2. Low-cost hybrid storage.
3. Access data with low latency.

6. AWS Backup

- Helps manage data backups across services such as:
 - EC2.
 - EBS.
 - EFS.
 - Create a **backup plan**.
 - The plan defines:
 1. **How frequently** to back up (frequency).
 2. **How long** to retain data (retention).
-

Source Page 10 - Content Delivery Network and CloudFront

Content Delivery Network (CDN)

- A mechanism to deliver content **quickly and efficiently based on geographic location**.
- Emphasizes low latency and shorter response time.
- The request can be served from a **distribution cache** rather than repeatedly reaching the origin.
- Origin examples drawn / written:
 - S3.
 - Elastic Load Balancer.
 - EC2.

CDN use cases written

1. S3 static website - deploy content globally.
2. Prevent attacks such as **DDoS**.
3. **IP address blocking** - prevent users in certain countries from accessing content.

1. Amazon CloudFront

- A CDN that delivers data and applications with **low latency**.
 - Makes content available globally or restricts it based on location.
 - Supports static or dynamic web content; the source emphasizes it is **mainly for static content**.
 - Uses **edge locations** to cache content.
 - Cache is described as a copy of a file.
 - Edge location is described as a location / data-center-like point where files are cached.
 - A collection of edge locations is referred to as a **distribution / cache distribution** in the notes.
-

Source Page 11 - Global Accelerator, Transfer Acceleration, VPC

Globally distributed edge locations - source note

- To make delivery global, use **globally distributed edge locations**.
- When you have many customers around the world, content can be uploaded to a **central bucket** and distributed.

2. Amazon Global Accelerator

- Mainly used for applications using **EC2 or Network Load Balancer**.
- Helps direct users to a dynamic application.
- Sends users through the **AWS global network** when accessing content.
- Speeds delivery by routing traffic to the **nearest healthy endpoint** based on:
 - Health checks.
 - Routing policies.
- Source note: intended to improve performance, with a handwritten reference around **60%** performance improvement.
- Uses router / routing behavior across the AWS global network rather than ordinary region-only routing.

3. S3 Transfer Acceleration

- Improves content **upload and download from S3 buckets**.
- Used for **fast file transfer over long distances**.

Networking - 1. VPC (Virtual Private Cloud)

- Allows you to create a **secure private network in AWS Cloud** where you launch resources.
 - You define:
 - IP ranges.
 - Subnets.
 - Security Groups.
 - Route tables.
 - Provides **isolation and protection** for resources.
 - A VPC spans **Availability Zones in a Region**.
 - Source explanation of AZ: one or more physically separated data centers with connectivity housed in separate facilities.
-

Source Page 12 - VPC Subcomponents, Internet Gateway, VPC Peering, NACL, Route Tables

VPC subcomponents

1. Subnet

- Allows you to **split the network inside the VPC**.
- Subnet is the place where you **launch resources**.

Private subnets

- Resources that should not be directly accessible from the Internet.
- Examples written:
 - Backend resources.
 - Database.

Public subnets

- Accessible **from the Internet**.
- Public access is achieved through related routing/network components.

Network ACL (NACL)

- Controls / filters traffic entering or leaving a subnet.
- Source notes indicate allowed traffic decisions at the subnet level.

2. Router & Route Table

- Define **where network traffic is directed**.

3. Internet Gateway

- Allows **public traffic / Internet connectivity** for the VPC.
- The source diagram connects:
 - Public subnet.
 - NACL.
 - Router / route table.
 - Internet Gateway.
 - Internet.
- Source note: the subnet is attached to a route table, and routing to the Internet uses the IGW.

VPC Peering

- Connects **two VPCs together via AWS private network**.
 - Makes them behave as one connected/private environment.
 - Source diagram shows VPC A ↔ peering connection ↔ VPC B.
 - Notes mention use in cloud and hybrid/network designs.
-

Source Page 13 - Route 53, API Gateway, Direct Connect, VPN

VPC Networking Services

2. Amazon Route 53

- A **DNS service**.
- DNS directs Internet traffic by connecting **domain names with web servers**.
- Route 53 routes users to the application.
- Can perform **domain-name registration**.
- Source explanation: the user accesses the server/application via a domain name rather than directly using an IP.
- Supports **health checks on AWS resources**; if a server/resource is unavailable, traffic can be routed accordingly.

3. Amazon API Gateway

- Allows systems to **share and manage data by creating APIs**.
- Can integrate with other services such as **Lambda**.
- Source diagram: Client ↔ API Gateway ↔ Lambda ↔ RDS.

1. AWS Direct Connect

- A **dedicated physical network connection** from your on-premises data center to AWS.
- Source emphasizes:
 - Very fast transfer.
 - Uses AWS private network connectivity rather than ordinary public Internet connectivity.
 - Can be used with VPC and compared with / used alongside peering concepts.

Direct Connect use cases

1. Transfer **large datasets**.
2. Transfer **business-critical data**.
3. Use in a **hybrid model**.

2. Site-to-Site VPN

- Connection **through the Internet**.
 - Data is **automatically encrypted**.
 - Source comparison: less dedicated than Direct Connect but provides secure connectivity through public Internet infrastructure.
-

Source Page 14 - Databases: RDS, Aurora, DynamoDB, Read Replicas

Databases

- Described as an **organized collection of various forms of data used by many applications**.

1. Amazon RDS - Relational Database Service

- Service that launches and manages relational databases.
- Supports popular database engines listed in the notes:
 - Aurora.
 - PostgreSQL.
 - MySQL.
 - MariaDB.
 - Oracle.
 - SQL Server.
- Benefits written:
 - Fault tolerance.
 - High availability.
 - **Multi-AZ** deployment option.
 - AWS handles:
 - Software patching.
 - Automated backups.
 - OS maintenance.
- Source note: you can launch **read replicas across regions**.

Read replicas

- A replica is described as a **read-only copy of the database for fast querying**.
- Can improve performance and durability / distribution across regions.

2. Amazon Aurora

- Relational database.
- Compatible with:
 - MySQL - source note says **5x faster than normal**.
 - PostgreSQL - source note says **3x faster than normal**.
- Source notes:
 - Cheaper (as written).
 - Scales automatically when needed.
 - Managed by RDS.

3. Amazon DynamoDB

- Fully managed **NoSQL / key-value database**.
 - **Serverless**.
 - Scales automatically.
 - Fast performance - the source highlights **milliseconds**.
 - Source note contrasts it with RDS, saying you do not choose/manage the same server / instance-type model.
-

Source Page 15 - Database Services and Real-World Use Cases, DMS

Real-world use cases for database services

1. Migrate on-premises **Oracle DB to cloud** → **RDS**.
2. Migrate on-premises **PostgreSQL DB to cloud** → **Aurora or RDS**.
3. Alleviate database load for data accessed often → **ElastiCache**.
4. Process large sets of user profiles and social interactions → **Neptune**.
5. NoSQL database fast enough to handle millions of requests per second → **DynamoDB**.
6. Operate MongoDB workloads at scale → **DocumentDB**.

4. Amazon DocumentDB

- Fully managed **document database**.
- Supports **MongoDB**-style/document workloads.
- Source notes:
 - Serverless / fully managed wording is used.
 - Different from key-value NoSQL model.

5. Amazon ElastiCache

- Fully managed **in-memory data store**.
- Compatible with:
 - Redis.
 - Memcached.
- In-memory datastores are used to **cache and accelerate data retrieval in applications**.
- Source warning: data in memory can be lost, so it is not the same as durable primary storage.
- Offers **high performance + low latency**.

6. Amazon Neptune

- Fully managed **graph database**.
- Supports highly connected datasets.
- Example written: **social-media networks**.
- Source notes emphasize fast and reliable operation.

Migration and Transfer - 1. AWS Database Migration Service (DMS)

- Helps migrate databases to AWS.
 - Source explanation includes migration between:
 - Homogeneous database engines.
 - Heterogeneous database engines.
 - Can also be used for Oracle-to-Oracle and different-engine scenarios as described in the notes.
-

Source Page 16 - DMS Continued, Server Migration Service, Snow Family

DMS - continuation

- Migration can be performed using **continuous data replication**.
- Source note: can provide **no downtime** / continue replication while moving database data.

DMS real-world use cases written

1. Oracle on-premises → **Aurora MySQL on cloud**.
2. Oracle on-premises → **Oracle on EC2**.
3. RDS Oracle → **Aurora MySQL** (cloud-to-cloud migration is noted).

2. Server Migration Service (SMS)

- Allows migration of a **group of servers from on-premises to cloud**.
- Source flow:
 1. Save the servers as **Amazon Machine Images (AMIs)**.
 2. Launch EC2 instances using those AMIs.
 3. Servers become cloud instances.

3. AWS Snow Family

- Used for transferring a **large amount of data** when network transfer may take too long / be too expensive.
- Allows transfer from on-premises to cloud using a **physical device**.
- Source notes compare it with Internet transfer when Internet speed/cost is not practical.

1. Snowcone

- Smallest member.
- Source note: approximately **8 TB** capacity.
- Can be used for offline shipping or **online using DataSync**.

2. Snowball & Snowball Edge

- Used for **petabyte-scale** transfer.
 - Transfer data physically from on-premises to cloud.
 - Snowball Edge includes **computing capability**, with support noted for **EC2 and Lambda**.
 - Useful when data must be processed locally before/while transferring.
 - Source note: can be cheaper than Internet transfer for some large-data cases.
-

Source Page 17 - Snowmobile, DataSync, Data Warehouse, Redshift

3. AWS Snowmobile

- Used for extremely large transfers in the **multi-petabyte or exabyte** range.
- Supports completely shutting down an on-premises DB/data environment and moving totally to the cloud.
- Source notes:
 - Data transferred securely.
 - GPS tracking / alarm monitoring.
 - **24/7 video surveillance.**

4. AWS DataSync

- Transfers data **online from on-premises to storage services in the cloud** such as:
 - S3.
 - EFS.
- Source states transfer speeds can be up to **10 times faster than open-source tools.**
- Can copy data:
 - Over the Internet.
 - With **Direct Connect.**
- Can also copy data **between AWS storage services.**
- Source note: this is copy/transfer rather than the same full migration model as shutting down the source.

Analytics - Data Warehouse

- A data-storage solution that aggregates a **massive amount of data from disparate sources**.
- Mainly used for:
 - Querying.
 - Reporting.
 - Analytics.
- Source comparison: unlike online transaction processing, a data warehouse is not primarily for frequent insert/update/delete transaction processing.

1. Amazon Redshift

- A **scalable data warehouse solution**.
- Can handle **exabyte-scale data** (as written in the source).
- Improves speed and efficiency when querying.

Redshift real-world use cases

1. **Data consolidation** - consolidate multiple data sources for reporting.
2. Use a relational store where you do not require real-time transaction processing (insert/update/delete).

Data replication note

- Source mentions replication patterns such as **cross-region or cross-account**.
-

Source Page 18 - EMR, Data Pipeline, Athena, Glue, Kinesis

Analytics

- Defined as the act of **querying or processing data**.

5. Amazon EMR - Elastic MapReduce

- Managed **big-data platform**.
- Includes popular ETL / big-data tools such as:
 - Apache Spark.
 - Apache Hadoop.
- Used for processing a **large amount of data**.
- Notes mention:
 - Data mining.
 - Machine learning.

6. AWS Data Pipeline

- Service that automates **movement and data transformation between various services**.
- Can work with **on-premises data sources** as well.
- Described as orchestration for data.
- Source capabilities / examples:
 - Move data at specific intervals.
 - Move data based on conditions.
 - Send notifications on success or failure.
 - Automate movement between services and sources.

2. Amazon Athena

- **Query service for S3**.
- Analyze data in S3 using **SQL**.
- Source notes:
 - **Pay per query**.
 - **Serverless**.

3. AWS Glue

- Prepares data for analytics; source mentions cleansing/understanding/transforming data.
- An **ETL service**.
- ETL = **Extract, Transform, Load**.
- Used to prepare/move data into a warehouse / analytics workflow.

4. Amazon Kinesis

- Analyze data in **real time** from videos and streams.
 - Real-time examples written:
 - Video + audio.
 - Application logs.
 - Website clickstreams.
 - IoT.
-

Source Page 19 - Comprehend, Polly, QuickSight, Rekognition

2. Amazon Comprehend

- **Natural Language Processing (NLP)** service.
- Finds insights and relationships to analyze text.

Comprehend - real-world example

- Review social-media posts.
- Search social-media posts for words / meaning / relationships.
- Source Arabic note emphasizes understanding the relationship between words and context, not only literal word matching.
- Used with **natural languages**.

3. Amazon Polly

- Turns **text to speech**.
- Mimics natural-sounding human speech.
- By default provides **several voices** and access to many languages.
- Can create a **custom voice**.
- Source note: output can be stored / used as **MP3 audio**.

7. Amazon QuickSight

- Helps **visualize your data**.
- Build **interactive dashboards**.
- Dashboards can be embedded into an application.

Analytics real-world use cases

1. Search/query data in S3 → **Athena** using SQL.
2. Log analytics / real-time analytics → **Kinesis** for application monitoring and fraud detection.

Machine Learning - Amazon Rekognition

- Analyze images and videos.
 - Identify custom labels.
 - Source notes include:
 - Face detection.
 - Text detection in images and videos.
 - Object detection.
-

Source Page 20 - Cloud9, CodeCommit, CodeBuild, SageMaker, Translate, Lex

Developer Tools - 1. AWS Cloud9

- **Integrated Development Environment (IDE) within your browser.**
- Write and debug code in the browser.

Cloud9 real-world use case

- Build a serverless application.
- Source notes that Cloud9 can be preconfigured with required **development SDKs / environment.**
- Write code in the browser.
- Source note describes importing / working with Lambda code from a local IDE / environment.

2. AWS CodeCommit

- Source-control system for **Git repositories** (compared with GitHub in the notes).
- Allows you to:
 - Create repositories.
 - Commit code.
 - Branch.
 - Merge code.
 - Collaborate with other developers.

3. AWS CodeBuild

- Build and test application source code.
- Compile code / use compiler.
- Run test cases.
- Enables CI/CD by creating a build artifact.
- Source note describes the artifact as the **deployable version produced by the compilation/build process.**

4. Amazon SageMaker

- Helps build, train, and deploy **machine-learning models**.
- Workflow written:
 1. Prepare data for models.
 2. Train and deploy the model.
 3. Provides deep-learning **AMIs / EC2 instances with high compute capacity** to accelerate ML/deep learning.

SageMaker - real-world example

- Make a **recommendation engine using ML models**.

5. Amazon Translate

- Provides **real-time and batch language translation**.
- Example written: **website localization**.

6. Amazon Lex

- Helps build **conversational interfaces / chatbots**.
 - Understands and recognizes speech / language.
 - Source note: integrates voice into a device.
 - Described as using the same underlying technology/power associated with **Amazon Alexa**.
-

Source Page 21 - X-Ray, CodeDeploy, CodePipeline, CodeStar

6. AWS X-Ray

- Used to **debug a production application and analyze it**.
- Performs **application component mapping**.
- Lets you view requests **end to end** across application components.

X-Ray - real-world example

- Map requests made to **RDS from your application**.
- Use traces / queries to see information and analyze/debug interactions between application components.

4. AWS CodeDeploy

- Manages deployment of code on:
 - Compute services.
 - On-premises environments.
- Helps maintain **application uptime / availability** while deploying.
- Supports rolling out a **new version of an application**.

CodeDeploy real-world example

- Run tests before deploying a new version of a production system (written next to CodeBuild/CodeDeploy workflow in the notes).

5. AWS CodePipeline

- Automates the **software release process**.
- Quickly delivers new features and updates.
- Integrations written:
 1. **CodeBuild** to run builds and unit tests.
 2. **CodeCommit** to receive source code.
 3. **CodeDeploy** to deploy changes.

CodePipeline - real-world example

- Automate **build, test, and deployment** of an application.
- Source highlighted note: implements DevOps practices that automate movement of code to production.

7. AWS CodeStar

- Source describes it as a tool used in the **CI/CD process** and working with CodeBuild, CodeCommit, CodeDeploy, etc.
 - Purpose in the notes: **simplify the development process**.
 - Supports developer collaboration on projects.
 - Includes an **issue-tracking dashboard**.
 - Source summary: **manage development pipeline**.
-

Source Page 22 - Elastic Beanstalk and CloudFormation / IaC

AWS Elastic Beanstalk

- Allows you to deploy **web applications and web servers to AWS**.
- Automatically handles deployment tasks such as:
 - Capacity provisioning.
 - Load balancing.
 - Auto scaling.
- Also monitors application health through a **health dashboard**.

Elastic Beanstalk - real-world example

- Quickly deploy a **Java-based web app** to AWS.
- After uploading Java code, Elastic Beanstalk handles capacity provisioning, load balancing, auto scaling, and health monitoring.
- Source note: automatic generation/provisioning of compute resources is handled for you.

Deployment & Infrastructure Management - Infrastructure as Code (IaC)

- Allows you to write **scripts to provision resources**.
- Benefits: **reproducible** infrastructure that can be created several times consistently.
- Source notes scripts/templates can be written using **JSON or YAML**.

1. AWS CloudFormation

- Provisions AWS resources using a **scripted template (IaC)**.

CloudFormation - real-world example

- Automate creation of **EC2 instances** in an AWS account.

2. Elastic Beanstalk vs CloudFormation - source comparison

- **Elastic Beanstalk** is designed for developers who want to write code and **not manage infrastructure**.
 - **CloudFormation** focuses on **Infrastructure as Code / infrastructure building**.
 - Source explanation: Beanstalk offers a simpler interface focused mainly on deployment; CloudFormation gives direct scripted infrastructure creation behind the scenes.
-

Source Page 23 - OpsWorks, SQS, SNS, Messaging & Integration

3. AWS OpsWorks

- Works with **Chef and Puppet**.
- Automates server configuration **using code**.
- Can be used for on-premises and cloud/EC2 server environments as described in the notes.

OpsWorks - real-world example

- Define software-installation scripts and automate configurations for servers.
- Source summary: automate **infrastructure configuration + management** for your application.

Messaging and Integration

- **Loose coupling** helps reduce the risk of cascading failures between components.
- Source notes refer to microservices and using **queues** to implement loosely coupled systems.

1. Amazon SQS - Simple Queue Service

- A **message queuing service**.
- Allows you to build **loosely coupled systems**.
- Enables component-to-component communication via messages.
- Messages are processed in an **asynchronous manner**.
- Source diagram/note: one process can continue rather than directly waiting for another process/component.
- Requests processed in **FIFO order** are noted.

SQS - real-world example

- Build a **money-transfer application** that performs well under heavy load.

2. Amazon SNS - Simple Notification Service

- Sends emails and text messages from an application and **publishes to a topic**.
- **Subscribers** receive the messages.

SNS - real-world example

- Send an email when **CPU utilization is above 80%**.
 - Source flow: CloudWatch Alarm → SNS → Email.
-

Source Page 24 - SES and CloudWatch

3. Amazon SES

- Allows sending **richly formatted HTML emails** from an application.
- Described as an ideal choice for:
 - Marketing campaigns.
 - Professional emails.

SES - real-world example

- Send marketing email and **track open / click-through rates**.

Auditing, Monitoring and Logging - 1. Amazon CloudWatch

- A **collection of services** that helps monitor and observe cloud resources.
- Collects:
 - Metrics.
 - Logs.
 - Events.
- Helps detect anomalies in the environment.
- Set alarms and visualize logs.

CloudWatch service breakdown written in the notes

1. CloudWatch Alarms

- Set high-resolution alarms.
- Trigger based on metric thresholds.
- Billing alarms are mentioned as an example.

2. CloudWatch Logs

- Monitor application logs / application performance logs.

3. CloudWatch Metrics

- Visualize time-series data, e.g., **CPU usage**.

4. CloudWatch Events

- Trigger an event based on a condition.

CloudWatch - real-world example

- Real-time monitoring on EC2.
 - Example flow: Instance CPU utilization > 80% → CloudWatch Alarm → SNS → Email.
-

Source Page 25 - CloudTrail, WorkSpaces, Amazon Connect

2. AWS CloudTrail

- Tracks **user activity and API calls**.
- Works across access methods such as:
 - Console.
 - SDK.
 - CLI.
- Used to:
 - Track activity.
 - Identify **which user made changes**.
 - Detect unusual activity in the account.
 - Log and retain account activity.

CloudTrail - real-world example

- Troubleshoot events over the past **90 days** using **CloudTrail Event History**.
- Find a specific event by time / region.

Things you can track with CloudTrail

1. User name.
2. Event time and name.
3. IP address.
4. Access key used.
5. Region.
6. Error code.

CloudWatch event example with root user

- Receive notification when **root-user activity** is detected in the account.
- Source scenario:
 - Root user makes API calls / activity.
 - Create a trigger/rule with **CloudWatch Event Rule**.
 - CloudWatch Event → SNS → Email.

Additional Services - 1. Amazon WorkSpaces

- Hosts a **virtual desktop in the cloud**.
- Windows or Linux desktops.
- Enables employees to **work from home**.

2. Amazon Connect

- Build a **cloud contact center / help desk in the cloud**.
 - Provides customer-service functionality.
 - Improves productivity of help-desk agents.
-

End of Domain 3 Notes

This document contains the clean written content of all **25 source pages** in the uploaded Domain 3 file, organized by source page and service.